

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-a2943035cba0bfff96.jsonl`  
- SHA-256: `de148687a85ecb1894b8718f6a45d945fbbcdc662a7f5e6ff3d684b51975461e`  
- Source modified: `2026-06-13T19:15:34+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_095035a0-e9f`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T19:13:53.204Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title":"Pixel-space colocation assumes the WASB/TrackNet trajectory and the cv2 person boxes share the same resolution ? undocumented coupling","severity":"low","file":"backend/pipeline/detectors/fusion_features.py:154-185 (_shuttle_in_box / shuttle_player_colocation); fusion_hybrid.py:99-106","detail":"shuttle_player_colocation compares the shuttle point (p.x, p.y) directly against person box pixel coords with a pad of pad_frac*frame_width / pad_frac*frame_height. The shuttle coords come from the WASB/TrackNet CSV (tracknet_runner.parse_trajectory_csv: x=float(row['X']), y=float(row['Y'])) ? raw pixels in the resolution WASB inferred on), and the person boxes come from cv2 reading the ORIGINAL video at CAP_PROP_FRAME_WIDTH/HEIGHT. The fusion math is only correct if WASB ran on the SAME resolution as the cv2 frames. Today wasb_runner.run_predict extracts frames from the staged ORIGINAL video with no downscale flag (wasb_runner.py:144-152), so they DO match ? but the memory notes a 'proxy-index' recipe where WASB runs on a downscaled proxy while the person detector reads the full-res original. In that mode the two pixel spaces diverge and colocation/_shuttle_in_box silently produce garbage (no scale conversion, no assertion). This is currently latent (not triggered by the default path) but is a coordinate-space landmine worth a guard or at least an explicit documented invariant. Note this only affects colocation; the foot/center features and two_sided_motion are all normalised by the cv2 frame dims so they are internally consistent regardless.","fix":"Document the hard invariant ('shuttle trajectory pixel space MUST equal the cv2 person-box pixel space') at the top of shuttle_player_colocation and/or in _person_features, and ideally normalise the shuttle point to [0,1] by the same fw/fh used for the boxes before the in-box test (then _shuttle_in_box also works in normalised space) so a future proxy-resolution shuttle run cannot silently break it."}

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:13:56.029Z)

I'll verify this finding by reading the actual code. Let me start by examining the key files

mentioned.

3. user (2026-06-13T19:13:57.008Z)

```
1      """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3      NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4      math is unit-testable in a GPU-free dev container. The detection (person boxes,
5      shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6      module only turns those signals into a handful of scale/translation-invariant
7      features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8      memorise this court/camera and fail to generalise off a small labelled set).
9
10     Conventions:
11     - A person box is `(x1, y1, x2, y2)` in pixels; `(track_id, box)` when motion
needs it.
12     - per_frame is `{frame_idx: [(track_id, box), ...]}` keyed by the ORIGINAL video
frame
13         index (so it aligns directly with the shuttle trajectory's frame ? no clip re-
timing).
14     - A shuttle point has frame, visible, x, y (pixels) ? the WASB
trajectory shape.
15     - court_polygon is normalised 0-1 [(x, y), ...] or None (None ? no mask,
every
16         detection counts ? the player-count-only mode).
17     - net is (axis 'x'|'y', position 0-1 normalised) or None (None ? two-sided =
0.0).
18
19     Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20     floats in [0, 1] except mean_player_motion (a non-negative normalised speed).
21     """
22     from __future__ import annotations
23
24     from typing import Dict, List, Optional, Sequence, Tuple
25
26     BBox = Tuple[float, float, float, float]
27     Net = Tuple[str, float]
28
29
30     def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31         """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32         (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33         if polygon is None or len(polygon) < 3:
34             return False
35         x, y = point
36         inside = False
37         n = len(polygon)
38         j = n - 1
39         for i in range(n):
40             xi, yi = polygon[i]
41             xj, yj = polygon[j]
42             # Does the horizontal ray at y cross edge (i, j)?
43             if (yi > y) != (yj > y):
44                 x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                 if x < x_cross:
46                     inside = not inside
47             j = i
48         return inside
49
50
51     def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52         """Normalised foot point (bottom-centre) of a person box ? where the player stands,
```

```

53         the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54         if frame_width <= 0 or frame_height <= 0:
55             return (0.0, 0.0)
56         x1, _y1, x2, y2 = box
57         return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60     def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61         if frame_width <= 0 or frame_height <= 0:
62             return (0.0, 0.0)
63         x1, y1, x2, y2 = box
64         return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70         True (count everyone) when it is None."""
71         if court_polygon is None:
72             return True
73         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
74
75
76     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
77                        frame_height: float,
78                        court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
79         """Per-frame count of in-court persons (one entry per sampled frame)."""
80         return [
81             sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
82             for _frame_idx, dets in sorted(per_frame.items())
83         ]
84
85
86     def _median(vals: Sequence[float]) -> float:
87         if not vals:
88             return 0.0
89         s = sorted(vals)
90         n = len(s)
91         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
92
93
94     def players_in_court(counts: Sequence[int]) -> float:
95         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
96         return _median(list(counts))
97
98
99     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
100         """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
101         if not counts:
102             return 0.0
103         return sum(1 for c in counts if c >= min_players) / len(counts)
104
105
106     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
107                           frame_width: float, frame_height: float) -> float:
108         """Mean per-track centre displacement between consecutive sampled frames, normalised
109         by the frame width (resolution-invariant). 0.0 if <2 frames or no shared tracks."""
110         if frame_width <= 0 or len(per_frame) < 2:
111             return 0.0
112         frames = sorted(per_frame.keys())
113         steps: List[float] = []

```

```

114     for a, b in zip(frames, frames[1:]):
115         prev = {tid: box for tid, box in per_frame[a]}
116         for tid, box in per_frame[b]:
117             if tid in prev:
118                 cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
119                 cx1, cy1 = _center_norm(box, frame_width, frame_height)
120                 steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
121     if not steps:
122         return 0.0
123     return sum(steps) / len(steps)
124
125
126 def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
127                     frame_height: float, net: Optional[Net],
128                     court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
129     """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
130
131     A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
132     The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
133     0.0.
134     """
135     if net is None or not per_frame:
136         return 0.0
137     axis, pos = net
138     both = 0
139     for _frame_idx, dets in per_frame.items():
140         near, far = False, False
141         for _tid, box in dets:
142             if not person_in_court(box, frame_width, frame_height, court_polygon):
143                 continue
144             fx, fy = _foot_norm(box, frame_width, frame_height)
145             coord = fx if axis == "x" else fy
146             if coord < pos:
147                 near = True
148             else:
149                 far = True
150             if near and far:
151                 both += 1
152     return both / len(per_frame)
153
154 def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
155 float,
156                     pad_frac: float) -> bool:
157     """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
158     width/height (the 'adjacent / in-hand' tolerance)?"""
159     x1, y1, x2, y2 = box
160     px = pad_frac * frame_width
161     py = pad_frac * frame_height
162     return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
163
164 def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
165 shuttle_points,
166                             frame_width: float, frame_height: float,
167                             pad_frac: float = 0.03) -> float:
168     """Fraction of (visible shuttle frames that were also person-sampled) in which the
169     shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
170     ~0**
171     ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
172     """
173     if not per_frame or frame_width <= 0 or frame_height <= 0:
174         return 0.0
175     aligned = 0
176     colocated = 0

```

```

175     for p in shuttle_points or []:
176         if not getattr(p, "visible", False):
177             continue
178         dets = per_frame.get(int(p.frame))
179         if dets is None:             # shuttle frame wasn't person-sampled ? can't judge
180             continue
181         aligned += 1
182         if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
183             for _tid, box in dets):
184             colocated += 1
185     return colocated / aligned if aligned else 0.0
186
187
188     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
189     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
190     PERSON_FEATURE_NAMES = [
191         "players_in_court", "two_sided_motion", "mean_player_motion",
192         "in_court_ratio", "shuttle_player_colocation",
193     ]
194
195
196     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
197 shuttle_points,
198                                     frame_width: float, frame_height: float,
199                                     court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
200                                     net: Optional[Net] = None,
201                                     min_players: int = 2) -> Dict[str, float]:
202     """Aggregate the per-window person features from per-frame detections + the in-
window
203 shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
204     counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
205     return {
206         "players_in_court": players_in_court(counts),
207         "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
208         "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
209         "in_court_ratio": in_court_ratio(counts, min_players),
210         "shuttle_player_colocation": shuttle_player_colocation(
per_frame, shuttle_points, frame_width, frame_height),
211     }
212

```

4. user (2026-06-13T19:13:57.364Z)

```

1     """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3     Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:
7
8     - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9     GPU-free, computed from the trajectory we already have); and, only when the person cue
10    is explicitly enabled (`indexing.fusion.cue_flags.person`), the person-cue features
11    (`fusion_features`), persisting them into `candidate_segments.stats` for the learned
12    fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13    never imports torch / touches the GPU.
14    - ``_select_for_handoff`` ? optional served Gate A/B: when
`indexing.fusion.min_crossings`
15    (and, with the person cue, `min_players`) are raised above 0, sub-threshold windows
are
16    dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
17    **Persisted candidates remain the full superset** regardless, so the offline
experiment

```

```

18     harness can re-score any variant.
19
20 Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
21 nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
22 and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
23 segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
24 experiment harness (EXPERIMENT_HARNESS.md), never silently.
25 """
26 from typing import Any, Dict, List, Optional, Tuple
27
28 from backend.pipeline.segmenters.base import segmenter_registry
29 from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
30 from backend.pipeline.detectors.base import DetectorRunner
31 from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
32 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
33 from backend.pipeline.detectors import rally_gate
34 from backend.pipeline.detectors import fusion_features as ff
35
36
37 class FusionSegmenter(TrajectoryHybridSegmenter):
38     """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
39
40     family = "fusion"
41     display_label = "Fusion"
42
43     def __init__(self, db: Any, config: Any):
44         super().__init__(db, config)
45         # Built lazily only if the person cue is enabled (keeps torch/GPU off the
default path).
46         self._person: Optional[Any] = None
47
48     @property
49     def name(self) -> str:
50         return "fusion_hybrid"
51
52     @property
53     def description(self) -> str:
54         return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
55                 "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
56                 "AI provider sets semantic boundaries.")
57
58     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
59         substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
60         if substrate == "tracknet":
61             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
62             return TrackNetRunner(cfg), {
63                 "weights_path": cfg.weights_path,
64                 "queue_length": cfg.queue_length,
65                 "fusion_substrate": "tracknet",
66             }
67         wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
68         return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
69
70     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
71                                 frame_width: float, video_path: str,
72                                 idx_cfg: Dict[str, Any]) -> Dict[str, Any]:

```

```

73         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
74         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
75         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
76         # over the whole trajectory by rally_gate (camera-agnostic).
77         gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
min_crossings=0)
78         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
79
80         fcfg = idx_cfg.get("fusion", {})
81         if fcfg.get("cue_flags", {}).get("person", False):
82             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
83         return stats
84
85     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
86                         frame_width: float, video_path: str,
87                         fcfg: Dict[str, Any]) -> Dict[str, float]:
88         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
89         frame range and compute the scale/translation-invariant person features. Lazily
builds
90         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
91         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
92         from backend.pipeline.detectors.person_detector import PersonDetector
93         if self._person is None:
94             self._person = PersonDetector(self.config)
95
96         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
97         start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
98         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
99         fw = det["frame_width"] or frame_width
100        fh = det["frame_height"] or 0.0
101
102        poly = fcfg.get("court_polygon")
103        polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
104        net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
105        shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
106        return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
107                                         court_polygon=polygon, net=net)
108
109     def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                           window_stats: List[Dict[str, Any]],
111                           idx_cfg: Dict[str, Any]) -> List[int]:
112         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113         (0) keep every window ? identical to the substrate. Persisted candidates are
114         untouched (full superset)."""
115         fcfg = idx_cfg.get("fusion", {})
116         min_crossings = int(fcfg.get("min_crossings", 0))
117         min_players = int(fcfg.get("min_players", 0))
118         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120         keep: List[int] = []
121         for i, st in enumerate(window_stats):
122             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                 continue
124             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                 continue
126             keep.append(i)

```

```

127         return keep
128
129
130     segmenter_registry.register(FusionSegmenter)
131

```

5. assistant (2026-06-13T19:14:00.868Z)

The finding hinges on the claim that the WASB/TrackNet trajectory pixel space can diverge from the cv2 person-box pixel space. Let me verify the key claims by reading the WASB and TrackNet runners.

6. user (2026-06-13T19:14:02.281Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12    import os
13    import shlex
14    import logging
15    import subprocess
16    from dataclasses import dataclass
17    from typing import Dict, Any, Optional, Tuple
18
19    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20    # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21    from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22    from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
23
24    logger = logging.getLogger(__name__)
25
26    # Windows path to the inference wrapper we stage into WSL.
27    _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30    @dataclass
31    class WasbConfig:
32        """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33        distro: str = "Ubuntu"
34        repo_dir: str = "~/models/WASB-SBDT"
35        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36        conda_env: str = "wasb"
37        weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38        sport: str = "badminton"
39        wsl_stage_dir: str = "~/clips"
40        timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41        keep_frames: bool = False # retain staged video + frame cache after success (debug
only)
42        min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
off)
43
44    @classmethod
45    def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":

```



```

46         w = (idx_cfg or {}).get("wasb", {}) or {}
47         return cls(
48             distro=w.get("wsl_distro", "Ubuntu"),
49             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
50             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
51             conda_env=w.get("conda_env", "wasb"),
52             weights_path=w.get("weights_path",
53                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
54             sport=w.get("sport", "badminton"),
55             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
56             timeout_sec=int(w.get("timeout_sec", 1800)),
57             keep_frames=bool(w.get("keep_frames", False)),
58             min_free_gb=float(w.get("min_free_gb", 0.0)),
59         )
60
61
62     class WasbRunner(WslCommandMixin, DetectorRunner):
63         def __init__(self, cfg: WasbConfig):
64             self.cfg = cfg
65
66         def healthcheck(self) -> Tuple[bool, str]:
67             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
68             cmd = (f"conda activate {self.cfg.conda_env} && "
69                  f"python -c \"import torch; print('torch', torch.__version__, "
70                  f"'cuda', torch.cuda.is_available())\"")
71             try:
72                 res = self._wsl_bash(cmd)
73             except FileNotFoundError:
74                 return False, "`wsl` not found ? Windows + WSL2 required."
75             except subprocess.TimeoutExpired:
76                 return False, "WSL healthcheck timed out."
77             out = (res.stdout or "").strip()
78             if res.returncode != 0:
79                 return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
80             return True, out
81
82         def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
83             src_mnt = to_wsl_mnt_path(video_win_path)
84             dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
85             cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
86             try:
87                 res = self._wsl_bash(cmd)
88             except subprocess.TimeoutExpired:
89                 logger.error("Staging video into WSL timed out.")
90                 return None
91             if res.returncode != 0 or "OK" not in (res.stdout or ""):
92                 logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
93                 return None
94             return dest
95
96         def _stage_infer_script(self) -> bool:
97             """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
98             src_mnt = to_wsl_mnt_path(_INFER_WIN)
99             cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
100             res = self._wsl_bash(cmd)
101             return res.returncode == 0 and "OK" in (res.stdout or "")
102
103         def run_predict(self, video_win_path: str, output_win_dir: str,
104                        video_id: Optional[str] = None) -> Optional[str]:
105             """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
106

```

```

107 All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
108 two videos that share a filename (e.g. two ``match.mp4`` from different folders)
109 can never reuse each other's staged frames, resumable cache, or CSV.
110 """
111 # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
112 # translation: to_wsl_mnt_path passes relative paths through unchanged, so
113 # WSL resolved them against the inference cwd (~/models/WASB-SBDT/src) and
114 # the CSV landed inside WSL while Windows polled the repo-relative path.
115 output_win_dir = os.path.abspath(output_win_dir)
116 os.makedirs(output_win_dir, exist_ok=True)
117 # Normalize for cross-platform basename (tests use Windows paths on Linux).
118 video_win_path = str(video_win_path).replace("\\", "/")
119 base = os.path.basename(video_win_path)
120 stem, ext = os.path.splitext(base)
121 # Unique, content-derived key: same filename + different bytes -> different key.
122 key = f"{stem}__{video_id[:12]}" if video_id else stem
123 wsl_out_dir = to_wsl_mnt_path(output_win_dir)
124 expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
125
126 if not self._stage_infer_script():
127     logger.error("Could not stage wasb_infer.py into WSL.")
128     return None
129 staged_video = self._stage_video(video_win_path, f"{key}{ext}")
130 if staged_video is None:
131     return None
132 frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
133
134 # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
135 # WSL stage filesystem is already low so the user can clean up before it fills.
136 if self.cfg.min_free_gb > 0:
137     free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
138     if free is not None and free < self.cfg.min_free_gb:
139         logger.warning(
140             "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
extraction "
141             "may fill the disk ? consider running tools/cleanup_caches.py.",
142             free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
143
144 cmd = (
145     f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
146     f"python wasb_infer.py "
147     f"--video {wsl_tilde_quote(staged_video)} "
148     f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
149     f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
150     f"--sport {shlex.quote(self.cfg.sport)} "
151     f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}")
152 )
153 logger.info("Running WASB inference on %s ...", base)
154 try:
155     res = self._wsl_bash(cmd)
156 except subprocess.TimeoutExpired:
157     logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
158     return None
159 if res.returncode != 0:
160     logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
161     return None
162 if not os.path.exists(expected_csv_win):
163     logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
164     return None
165 logger.info("WASB trajectory CSV: %s", expected_csv_win)
166
167 # Success ? the trajectory CSV is the durable output; the staged video copy and
168 # the (huge) decoded frame cache are now pure intermediates. Delete them unless

```

```

169         # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
170         # so the frame cache + detector cache remain for a resumable re-run.
171         if not self.cfg.keep_frames:
172             logger.info("Cache hygiene: removing staged video + frame cache for %s "
173                         "(set indexing.wasb.keep_frames=true to retain).", key)
174             self._wsl_rm_rf([frames_dir, staged_video])
175         return expected_csv_win
176
177     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
178     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
179     trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
180

```

7. user (2026-06-13T19:14:02.536Z)

```

1     """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
3     Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4     the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5     `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
identically);
6     next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
7     later the learned classifier reads its persisted features. Keeping detection here (not
in
8     the segmenter) is what makes "add a cue = register a producer" true.
9
10    Behaviour is unchanged from the in-lined version ? only the home moved:
11    - `lazy_load_model`           ? load a permissively-licensed torchvision detector
12    - `make_tracker`             ? a fresh supervision ByteTrack per chunk
13    - `detect_and_track`         ? one frame ? confident persons with persistent IDs
14    - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
15    with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
16    that feed the candidate `stats` JSON and the learned fusion features.
17
18    Heavy deps (torchvision, supervision) are imported lazily inside the methods so
importing
19    the segmenter registry never pulls torch on a machine that lacks it (and tests can mock
20    the seams). Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
21    (ADR-005; ultralytics' AGPL YOLO was dropped).
22    """
23    import logging
24    from typing import Any, Dict, List, Optional, Tuple
25
26    import cv2
27    import torch
28
29    from backend.utils.geometry import get_velocity_normalised
30
31    logger = logging.getLogger(__name__)
32
33    # torchvision detection models are trained on COCO where the "person" category is
34    # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
35    # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
36    _PERSON_CLASS_ID = 1
37
38    # Permissively-licensed torchvision detectors selectable via config
39    # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
40    _DETECTOR_FACTORIES = {
41        "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
42        "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
43        "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
44    }
45    _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"

```

```

46
47
48 class PersonDetector:
49     """Stage-1 person detection + tracking + motion-window extraction (a rally cue).
50
51     Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
52     ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
53     min_action_window_duration}``); the model loads lazily on first use.
54     """
55
56     def __init__(self, config: Dict[str, Any]):
57         self.config = config
58         # Dynamic torchvision module (None until lazy_load_model); typed Any so the
59         # `self.model([tensor])` inference call type-checks without a quarantine.
60         self.model: Any = None
61         self.device: Optional[str] = None
62
63     def lazy_load_model(self) -> None:
64         if self.model is not None:
65             return
66
67         # Imported lazily so the module imports without torchvision present and so
68         # the test suite (which pre-sets self.model) never pulls in model weights.
69         from torchvision.models import detection as tv_detection
70
71         idx_cfg = self.config.get("indexing", {})
72         use_gpu = idx_cfg.get("use_gpu", True)
73         self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
74         if use_gpu and not torch.cuda.is_available():
75             logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
76
77         model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
78         if model_name not in _DETECTOR_FACTORIES:
79             logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
80             model_name = _DEFAULT_DETECTOR
81
82         builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
83         logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
84         # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
85         self.model = builder(weights="DEFAULT")
86         self.model.eval()
87         self.model.to(self.device)
88
89     def make_tracker(self, fps: float) -> Any:
90         """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
91
92         Imported lazily so tests can mock this method without supervision installed.
93         """
94         import supervision as sv
95         frame_rate = max(1, int(round(fps)))
96         return sv.ByteTrack(frame_rate=frame_rate)
97
98     def detect_and_track(self, frame: Any, tracker: Any,
99                         score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
100         """Detect persons in a frame and assign persistent track IDs.
101
102         Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
103         This replaces the old ultralytics ``model.track(...)`` call, which bundled
104         detection AND association: torchvision does the detection, supervision's
105         ByteTrack does the association.
106         """
107         import supervision as sv

```

```

108
109         # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
110         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
111         tensor = torch.from_numpy(rgb).permute(2, 0,
112 1).float().div(255.0).to(self.device)
113         with torch.no_grad():
114             outputs = self.model([tensor])
115
116         out = outputs[0]
117         boxes = out["boxes"].detach().cpu().numpy()
118         labels = out["labels"].detach().cpu().numpy()
119         scores = out["scores"].detach().cpu().numpy()
120
121         # Keep only confident person detections.
122         mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
123         if not mask.any():
124             return []
125
126         # ByteTrack requires confidence to be populated (it filters on it internally).
127         dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
128 class_id=labels[mask])
129         tracked = tracker.update_with_detections(dets)
130
131         result: List[Tuple[int, Tuple[float, float, float, float]]] = []
132         for i in range(len(tracked)):
133             tid = tracked.tracker_id[i]
134             if tid is None:
135                 continue
136             x1, y1, x2, y2 = tracked.xyxy[i]
137             result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
138         return result
139
140     def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
141 velocity_thresh: float, merge_gap: float,
142 frame_skip: int = 3,
143 chunk_index: Optional[int] = None) ->
144 List[Dict[str, Any]]:
145     """
146     Runs person detection + tracking on a video chunk and extracts high-motion
147     action windows. Returns a list of dicts in the full video's timeframe, each
148     with:
149         start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
150         track_id_count, chunk_index.
151
152     Args:
153         frame_skip: Process every Nth frame (1 = every frame). Higher values
154             dramatically reduce inference cost with minimal quality loss.
155         chunk_index: Index of the source chunk (recorded on each window for
156         traceability).
157     """
158     cap = cv2.VideoCapture(chunk_path)
159     if not cap.isOpened():
160         logger.error(f"Could not open {chunk_path}")
161         return []
162
163     fps = cap.get(cv2.CAP_PROP_FPS)
164     if fps <= 0:
165         fps = 30.0
166
167     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
168     if frame_width <= 0:
169         frame_width = 1280.0 # Sensible fallback
170
171     # Effective sampling rate after frame-skipping
172     effective_fps = fps / max(frame_skip, 1)

```

```

168
169 # A fresh tracker per chunk ? track IDs only need to be consistent within a
170 # chunk (windows are computed per chunk, then merged across chunks by time).
171 tracker = self.make_tracker(effective_fps)
172 score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
173
174 # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
175 # so windows can be enriched with motion stats for logging + fine-tuning.
176 active_frames = []
177 frame_idx = 0
178 track_history: Dict[int, Tuple[float, float, float, float]] = {}
179
180 while True:
181     ret, frame = cap.read()
182     if not ret:
183         break
184
185     # Skip frames for performance ? at 30fps, every 3rd frame still
186     # gives ~10 samples/second, plenty for human-scale velocity.
187     if frame_idx % max(frame_skip, 1) != 0:
188         frame_idx += 1
189         continue
190
191     # Detect persons + assign persistent track IDs.
192     detections = self.detect_and_track(frame, tracker, score_thresh)
193
194     frame_peak_velocity = 0.0
195     frame_active_ids = set()
196     current_ids = set()
197
198     for track_id, bbox in detections:
199         current_ids.add(track_id)
200
201         if track_id in track_history:
202             prev_bbox = track_history[track_id]
203             velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
204             if velocity > velocity_thresh:
205                 frame_peak_velocity = max(frame_peak_velocity, velocity)
206                 frame_active_ids.add(track_id)
207
208             track_history[track_id] = bbox
209
210     # Prune stale IDs that are no longer visible ? prevents false
211     # velocity spikes when the tracker reassigns an old ID.
212     track_history = {k: v for k, v in track_history.items() if k in current_ids}
213
214     if frame_active_ids:
215         active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
216
217     frame_idx += 1
218
219 cap.release()
220
221 # Group active frames into continuous windows
222 if not active_frames:
223     return []
224
225 max_gap_frames = int(merge_gap * fps)
226
227 def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
228     velocities = [v for _, v, _ in group]
229     track_ids = set().union(*[ids for _, _, ids in group])
230     return {

```

```

231         "start": group[0][0] / fps + chunk_start,
232         "end": group[-1][0] / fps + chunk_start,
233         "active_frame_count": len(group),
234         "peak_velocity": max(velocities),
235         "mean_velocity": sum(velocities) / len(velocities),
236         "track_id_count": len(track_ids),
237         "chunk_index": chunk_index,
238     }
239
240     windows = []
241     group = [active_frames[0]]
242     for af in active_frames[1:]:
243         if af[0] - group[-1][0] <= max_gap_frames:
244             group.append(af)
245         else:
246             windows.append(_finalize(group))
247             group = [af]
248     windows.append(_finalize(group))
249
250     # Filter out extremely short action windows
251     min_window_duration = self.config.get("indexing",
252     {}).get("min_action_window_duration", 2.0)
253     windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
254
255     return windows
256
257     def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
258     frame_skip: int = 3) -> Dict[str, Any]:
259         """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL
260         video.
261         Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
262         ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and
263         discards the
264         per-frame boxes), this returns the raw per-frame detections so the fusion
265         features can
266         align them with the shuttle trajectory. It reads the source by **absolute frame
267         index**
268         (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
269         match
270         the WASB trajectory's ``.frame`` exactly ? that alignment is what
271         ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
272         Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
273         "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-
274         seek
275         accuracy varies by codec (it lands near the requested keyframe then decodes
276         forward);
277         treat the indices as ±a few frames on real footage ? fine for window-level
278         features.
279         """
280         self.lazy_load_model()
281         cap = cv2.VideoCapture(video_path)
282         if not cap.isOpened():
283             logger.error(f"Could not open {video_path}")
284             return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
285
286         fps = cap.get(cv2.CAP_PROP_FPS)
287         if fps <= 0:
288             fps = 30.0
289         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
290         frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
291         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
292         0.5)
293         step = max(frame_skip, 1)

```

```

286         tracker = self.make_tracker(fps / step)
287
288         start_frame = max(0, int(start_frame))
289         frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
290         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
291         frame_idx = start_frame
292         while frame_idx <= end_frame:
293             ret, frame = cap.read()
294             if not ret:
295                 break
296             if (frame_idx - start_frame) % step == 0:
297                 frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
298             frame_idx += 1
299         cap.release()
300         return {"frames": frames, "fps": fps,
301               "frame_width": float(frame_width), "frame_height": float(frame_height)}
302

```

8. assistant (2026-06-13T19:14:05.327Z)

Now let me verify the WASB inference script (`wasb\_infer.py`) to confirm whether it extracts frames at original resolution or downscales ? this is the crux of the claim.

9. user (2026-06-13T19:14:06.782Z)

```

1      """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3      WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4      own building blocks (ImageDataset transform + detector + online tracker) to run on
5      any folder of extracted frames and emit a trajectory CSV in the exact shape the
6      indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8          Frame,Visibility,X,Y
9
10     It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11     imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12     (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14     Resumable across reboots
15     -----
16     The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17     the slow, expensive part; the CPU tracker pass that turns detections into a
18     trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19     resume mid-stream). We exploit that split:
20
21     * The detector pass writes its per-window raw outputs incrementally to
22       ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23       records progress in ``<cache>/manifest.jsonl`` (atomic write). A reboot loses at
24       most ``--flush-every`` windows of GPU work instead of the whole pass.
25     * On restart we replay the cached windows, skip the DataLoader ahead to the first
26       un-cached window, and continue. Once every window is cached, the detector pass
27       is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28       the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29       not the GPU hour.
30
31     The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32     to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33     weights, sport, window size, or frame count change.
34
35     Usage (in WSL, from ~/models/WASB-SBDT/src):
36         python wasb_infer.py --frames_dir /path/frames --weights
37         ../pretrained_weights/wasb_badminton_best.pth.tar \
38         --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-

```



```

39     import os
40     import os.path as osp
41     import csv
42     import glob
43     import json
44     import argparse
45     import logging
46     from collections import defaultdict
47
48     logger = logging.getLogger(__name__)
49
50     CACHE_VERSION = 1
51     _DET_FILE = "det_raw.jsonl"
52     _MANIFEST_FILE = "manifest.json"
53
54
55     def _build_cfg(weights: str, sport: str):
56         """Compose the WASB eval config via Hydra, overriding model/weights/device."""
57         from hydra import compose, initialize
58         with initialize(config_path="configs", version_base=None):
59             cfg = compose(config_name="eval", overrides=[
60                 f"dataset={sport}",
61                 "model=wasb",
62                 f"detector.model_path={weights}",
63                 "runner.device=cuda",
64                 "runner.gpus=[0]",          # this box has a single GPU; default config assumes
more
65             ])
66             return cfg
67
68
69     def _frame_id(path: str) -> int:
70         """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
180)."""
71         stem = osp.splitext(osp.basename(path))[0]
72         digits = "".join(ch for ch in stem if ch.isdigit())
73         return int(digits) if digits else -1
74
75
76     # ----- #
77     # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
78     # ----- #
79     def _cache_paths(cache_dir: str):
80         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
81
82
83     def default_cache_dir(out_csv: str) -> str:
84         """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
85         d = osp.dirname(osp.abspath(out_csv))
86         stem = osp.splitext(osp.basename(out_csv))[0]
87         return osp.join(d, stem + "_wasbcache")
88
89
90     def _atomic_write_text(path: str, text: str) -> None:
91         """Write then rename so a crash mid-write can't leave a half-written file."""
92         tmp = path + ".tmp"
93         with open(tmp, "w") as f:
94             f.write(text)
95             f.flush()
96             os.fsync(f.fileno())
97         os.replace(tmp, path)
98
99
100     def _det_plain(det) -> list:
101         """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able

```

```

list."""
102     xy = det["xy"]
103     return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
104
105
106 def _dets_to_tracker(plain_list):
107     """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
108     the tracker does vector math / np.linalg.norm on it)."""
109     import numpy as np
110     return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
111             for p in plain_list]
112
113
114 def write_manifest(cache_dir: str, manifest: dict) -> None:
115     _, mpath = _cache_paths(cache_dir)
116     _atomic_write_text(mpath, json.dumps(manifest, indent=2))
117
118
119 def read_manifest(cache_dir: str):
120     _, mpath = _cache_paths(cache_dir)
121     if not osp.exists(mpath):
122         return None
123     try:
124         with open(mpath) as f:
125             return json.load(f)
126     except (json.JSONDecodeError, OSError):
127         return None
128
129
130 def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
131                         frames_in: int, frames_total: int) -> bool:
132     """A cache is reusable only if the run that produced it matches this run's
inputs."""
133     if not manifest or manifest.get("version") != CACHE_VERSION:
134         return False
135     return (
136         manifest.get("frames_dir") == osp.abspath(frames_dir)
137         and manifest.get("weights") == weights
138         and manifest.get("sport") == sport
139         and manifest.get("frames_in") == frames_in
140         and manifest.get("frames_total") == frames_total
141     )
142
143
144 def reset_cache(cache_dir: str) -> None:
145     """Drop any existing cache so the next run starts clean."""
146     det_jsonl, mpath = _cache_paths(cache_dir)
147     for p in (det_jsonl, mpath):
148         if osp.exists(p):
149             os.remove(p)
150
151
152 def load_and_clean_cache(cache_dir: str):
153     """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
154     rebuild the per-frame detection accumulation, and rewrite the file to exactly that
155     prefix so a trailing half-written line from a crash can't corrupt future appends.
156
157     Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
158     [x, y, score, scale] candidates (accumulated across every window the frame is in,
159     in window order ? identical to a non-resumed run).
160     """
161     det_jsonl, _ = _cache_paths(cache_dir)
162     det_by_fid = defaultdict(list)
163     valid: list = []

```

```

164         if osp.exists(det_jsonl):
165             with open(det_jsonl, "r") as f:
166                 for line in f:
167                     s = line.strip()
168                     if not s:
169                         continue
170                     try:
171                         obj = json.loads(s)
172                     except json.JSONDecodeError:
173                         break # truncated trailing line (crash mid-
flush)
174                     if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
175                         break
176                     valid.append(s)
177                     for fid, plain in obj["f"]:
178                         det_by_fid[int(fid)].extend([list(p) for p in plain])
179             # Guarantee a clean append boundary by rewriting only the good prefix.
180             _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
181         return len(valid), det_by_fid
182
183
184 def _append_windows(fh, objs) -> None:
185     """Append window records as JSONL and durably flush (bounded loss on crash)."""
186     for o in objs:
187         fh.write(json.dumps(o, separators=(",", ":")) + "\n")
188     fh.flush()
189     os.fsync(fh.fileno())
190
191
192 def _atomic_write_trajectory(out_csv: str, rows) -> None:
193     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
194     tmp = out_csv + ".tmp"
195     with open(tmp, "w", newline="") as f:
196         w = csv.writer(f)
197         w.writerow(["Frame", "Visibility", "X", "Y"])
198         w.writerows(rows)
199         f.flush()
200     os.fsync(f.fileno())
201     os.replace(tmp, out_csv)
202
203
204 # ----- #
205 # Inference
206 # ----- #
207 def run(frames_dir: str, weights: str, out_csv: str, sport: str = "badminton",
208         limit: int = 0, batch_size: int = 8, num_workers: int = 4,
209         log_every_batches: int = 50, cache_dir: "str | None" = None,
210         flush_every: int = 1000, fresh: bool = False) -> str:
211     import time
212     import torch
213     from torch.utils.data import DataLoader
214     from detectors import build_detector
215     from trackers import build_tracker
216     from dataloaders import build_img_transforms, build_seq_transforms
217     from dataloaders.dataset_loader import ImageDataset
218     from utils import Center
219
220     cfg = _build_cfg(weights, sport)
221     frames_in = int(cfg["model"]["frames_in"])
222     input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
223     output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
224
225     frames = sorted(glob.glob(osp.join(frames_dir, "*.png"))) +
glob.glob(osp.join(frames_dir, "*.jpg")))
226     if limit:

```

```

227         frames = frames[:limit]
228     if len(frames) < frames_in:
229         raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in
{frames_dir}")
230
231     # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
232     samples = []
233     for i in range(len(frames) - frames_in + 1):
234         window = frames[i:i + frames_in]
235         annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
236         samples.append({"frames": window, "annos": annos})
237     windows_total = len(samples)
238
239     # ---- cache setup / resume decision ----- #
240     if cache_dir is None:
241         cache_dir = default_cache_dir(out_csv)
242         os.makedirs(cache_dir, exist_ok=True)
243         det_jsonl, _ = _cache_paths(cache_dir)
244         manifest = None if fresh else read_manifest(cache_dir)
245         resume = manifest_compatible(
246             manifest or {}, frames_dir=frames_dir, weights=weights, sport=sport,
247             frames_in=frames_in, frames_total=len(frames),
248         )
249         if resume:
250             windows_done, det_by_fid = load_and_clean_cache(cache_dir)
251             logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
252         else:
253             if manifest is not None:
254                 logger.info("cache incompatible with this run -> starting fresh")
255                 reset_cache(cache_dir)
256                 windows_done, det_by_fid = 0, defaultdict(list)
257
258     base_manifest = {
259         "version": CACHE_VERSION,
260         "frames_dir": osp.abspath(frames_dir),
261         "weights": weights,
262         "sport": sport,
263         "frames_in": frames_in,
264         "frames_total": len(frames),
265         "windows_total": windows_total,
266         "windows_done": windows_done,
267     }
268     write_manifest(cache_dir, base_manifest)
269
270     # ---- detector pass over the un-cached windows ----- #
271     remaining = samples[windows_done:]
272     if remaining:
273         _, transform_test = build_img_transforms(cfg)
274         try:
275             _, seq_transform_test = build_seq_transforms(cfg)
276         except Exception:
277             seq_transform_test = None
278         ds = ImageDataset(cfg, remaining, input_wh, output_wh,
279                         transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
280         loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=num_workers)
281         detector = build_detector(cfg)
282
283         t0 = time.time()
284         done = windows_done
285         win_idx = windows_done

```

```

286         log_every = max(1, log_every_batches)
287         flush_n = max(1, flush_every)
288         pending = []
289         logger.info(f"detecting on {len(remaining)} remaining windows ")
290         f"(total {windows_total}, batch={batch_size},
workers={num_workers})..."
291         det_f = open(det_jsonl, "a")
292         try:
293             with torch.no_grad():
294                 for bi, batch in enumerate(loader):
295                     imgs, _hms, trans = batch[0], batch[1], batch[2]
296                     img_paths = [list(t) for t in batch[-1]] # [frame_in][batch] ->
path
297                     batch_results, _ = detector.run_tensor(imgs, trans)
298                     nb = imgs.shape[0]
299                     for ib in range(nb):
300                         frames_payload = []
301                         for ie in sorted(batch_results[ib].keys()):
302                             p = img_paths[ie][ib]
303                             fid = _frame_id(p)
304                             plain = [_det_plain(d) for d in batch_results[ib][ie]]
305                             frames_payload.append([fid, plain])
306                             det_by_fid[fid].extend(plain)
307                         pending.append({"w": win_idx, "f": frames_payload})
308                         win_idx += 1
309                     done += nb
310                     if len(pending) >= flush_n or done >= windows_total:
311                         _append_windows(det_f, pending)
312                         pending = []
313                         base_manifest["windows_done"] = done
314                         write_manifest(cache_dir, base_manifest)
315                     if (bi + 1) % log_every == 0 or done >= windows_total:
316                         el = time.time() - t0
317                         new_done = done - windows_done
318                         rate = new_done / el if el > 0 else 0.0
319                         eta = (windows_total - done) / rate if rate > 0 else 0.0
320                         logger.info(f"{done}/{windows_total} "
321                                     f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
322                                     f"elapsed {el:.0f}s | ETA {eta:.0f}s")
323                     if pending:
324                         _append_windows(det_f, pending)
325                         base_manifest["windows_done"] = done
326                         write_manifest(cache_dir, base_manifest)
327                 finally:
328                     det_f.close()
329                 logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
330             else:
331                 logger.info("detector cache already complete -> tracker-only re-run")
332
333             # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
334             tracker = build_tracker(cfg)
335             tracker.refresh()
336             rows = []
337             for fid in sorted(det_by_fid.keys()):
338                 r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
339                 rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
340
341             _atomic_write_trajectory(out_csv, rows)
342             visible = sum(1 for _, v, *_ in rows if v)
343             logger.info(f"wrote {len(rows)} rows -> {out_csv}")
344             logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
345             return out_csv
346
347

```

```

348 def extract_frames(video_path: str, frames_dir: str) -> str:
349     """Decode a video to PNG frames named by source frame index (00000000.png ...).
350
351     Idempotent: if the dir already holds the expected number of frames (matching the
352     container's reported frame count) we skip re-decoding ? re-extraction after a
353     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
354     extracting with ffmpeg on the host and passing --frames_dir.
355     """
356     import cv2
357     os.makedirs(frames_dir, exist_ok=True)
358     cap = cv2.VideoCapture(video_path)
359     if not cap.isOpened():
360         raise SystemExit(f"cannot open video: {video_path}")
361     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
362     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
363     if expected > 0 and existing >= expected:
364         cap.release()
365         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
366         return frames_dir
367     i = 0
368     while True:
369         ok, frame = cap.read()
370         if not ok:
371             break
372         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
373         i += 1
374     cap.release()
375     logger.info(f"extracted {i} frames -> {frames_dir}")
376     return frames_dir
377
378
379 def main():
380     ap = argparse.ArgumentParser()
381     ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
382     ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
383     ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
384     ap.add_argument("--weights", required=True)
385     ap.add_argument("--out", required=True)
386     ap.add_argument("--sport", default="badminton")
387     ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
388     ap.add_argument("--batch-size", type=int, default=8)
389     ap.add_argument("--num-workers", type=int, default=4)
390     ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
391     ap.add_argument("--cache-dir", default=None,
392                     help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
393     ap.add_argument("--flush-every", type=int, default=1000,
394                     help="fsync the detector cache every N windows (bounds reboot
loss)")
395     ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
396     args = ap.parse_args()
397
398     frames_dir = args.frames_dir
399     if args.video:
400         frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
401     if not frames_dir:
402         raise SystemExit("provide --frames_dir or --video")
403     run(frames_dir, args.weights, args.out, args.sport, args.limit,
404         batch_size=args.batch_size, num_workers=args.num_workers,

```

```

405         log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
406         flush_every=args.flush_every, fresh=args.fresh)
407
408
409     if __name__ == "__main__":
410         main()
411

```

10. user (2026-06-13T19:14:07.062Z)

```

1     """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4     inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6         1. Translate the Windows video path to its WSL-visible form (and optionally
7           stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8         2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9         3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10        4. Convert that trajectory into high-motion "action windows" ? the same
11           candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12           pipeline (AI handoff, DB population) is unchanged.
13
14     Nothing here imports TensorFlow; all model code stays in WSL.
15     """
16
17     import os
18     import csv
19     import shlex
20     import logging
21     import subprocess
22     from dataclasses import dataclass
23     from typing import List, Dict, Any, Optional, Tuple
24
25     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27     logger = logging.getLogger(__name__)
28
29
30     @dataclass
31     class TrackNetConfig:
32         """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/TrackNetV4" # WSL path to the cloned repo
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "TrackNetV4"
37         weights_path: str = "" # WSL path to the .h5/.keras weights
38         (REQUIRED)
39         queue_length: int = 5
40         stage_in_wsl: bool = True # copy video into WSL fs first
41         (perf)
42         wsl_stage_dir: str = "~/clips" # where staged videos/outputs live
43         in WSL
44         timeout_sec: int = 1800 # 30 min; kills a hung call (0 = no
timeout)
45         keep_frames: bool = False # retain staged video after success
46         (debug only)
47         min_free_gb: float = 0.0 # warn if WSL free space below this
(0 = off)
48
49
50     @classmethod
51     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
52         tn = (idx_cfg or {}).get("tracknet", {}) or {}

```

```

48         return cls(
49             distro=tn.get("wsl_distro", "Ubuntu"),
50             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52             conda_env=tn.get("conda_env", "TrackNetV4"),
53             weights_path=tn.get("weights_path", ""),
54             queue_length=int(tn.get("queue_length", 5)),
55             stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56             wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57             timeout_sec=int(tn.get("timeout_sec", 1800)),
58             keep_frames=bool(tn.get("keep_frames", False)),
59             min_free_gb=float(tn.get("min_free_gb", 0.0)),
60         )
61
62
63 def to_wsl_mnt_path(win_path: str) -> str:
64     """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66     Already-POSIX paths (starting with / or ~) are returned unchanged so the
67     same helper is safe to call on either kind of path.
68     """
69     p = str(win_path)
70     if p.startswith("/") or p.startswith("~"):
71         return p
72     p = p.replace("\\", "/")
73     if len(p) >= 2 and p[1] == ":":
74         drive = p[0].lower()
75         return f"/mnt/{drive}{p[2:]}"
76     return p
77
78
79 @dataclass
80 class TrajectoryPoint:
81     frame: int
82     visible: bool
83     x: float
84     y: float
85
86
87 class TrackNetRunner(WslCommandMixin, DetectorRunner):
88     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90     Implements the common DetectorRunner contract so a future Linux-native or
91     alternative trajectory runner can be substituted without touching the hybrids.
92     """
93
94     def __init__(self, cfg: TrackNetConfig):
95         self.cfg = cfg
96
97     def healthcheck(self) -> Tuple[bool, str]:
98         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100         Returns (ok, message). Cheap to call before a long run.
101         """
102         cmd = (
103             f"conda activate {self.cfg.conda_env} && "
104             f"python -c \"import tensorflow as tf; "
105             f"print('TF', tf.__version__, 'GPUs', "
106             len(tf.config.list_physical_devices('GPU')))\\"
107         )
108         try:
109             res = self._wsl_bash(cmd)
110         except FileNotFoundError:
111             return False, "wsl` not found ? is this running on Windows with WSL2 installed?"

```



```

111         except subprocess.TimeoutExpired:
112             return False, "WSL healthcheck timed out."
113         out = (res.stdout or "").strip()
114         if res.returncode != 0:
115             return False, f"WSL env check failed: {(res.stderr or out).strip()}"
116         return True, out
117
118     # ----- inference
119
120     def run_predict(self, video_win_path: str, output_win_dir: str,
121                    video_id: Optional[str] = None) -> Optional[str]:
122         """Run predict.py on a video. Returns the Windows path to the produced CSV, or
123         None.
124
125         ``output_win_dir`` is a Windows directory (under the repo's output/) that
126         WSL writes into via its /mnt view, so the windows process can read the CSV
127         back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
128         therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
129         a filename cannot collide on the output CSV.
130
131         """
132         if not self.cfg.weights_path:
133             logger.error(
134                 "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
135                 "in config.json to the WSL path of your trained TrackNetV4 weights."
136             )
137             return None
138
139         # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
140         # relative paths through unchanged, so WSL would resolve them against the
141         # predict.py cwd instead of the repo (same bug class as wasb_runner).
142         output_win_dir = os.path.abspath(output_win_dir)
143         os.makedirs(output_win_dir, exist_ok=True)
144         # Normalize separators so basename/splitext work when tests (or collector)
145         # pass windows-style paths on a Linux host.
146         video_win_path = str(video_win_path).replace("\\", "/")
147         base = os.path.basename(video_win_path)
148         stem, ext = os.path.splitext(base)
149
150         # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
151         if ext.lower() not in (".mp4", ".avi"):
152             logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
153             return None
154
155         wsl_out = to_wsl_mnt_path(output_win_dir)
156         # Namespace by video_id so two same-filename videos don't collide on the CSV.
157         # predict.py derives its output name from the (staged) video stem, so staging
158         # under the namespaced name propagates into "<key>_predict.csv".
159         key = f"{stem}_{video_id[:12]}" if video_id else stem
160         expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
161
162         # Resolve the video path WSL will read.
163         if self.cfg.stage_in_wsl:
164             staged = self._stage_video(video_win_path, f"{key}{ext}")
165             if staged is None:
166                 return None
167             wsl_video = staged
168         else:
169             # No staging: predict.py reads /mnt directly and writes
170             # "<stem>_predict.csv".
171             # We cannot rename its output, so fall back to the un-namespaced name.
172             wsl_video = to_wsl_mnt_path(video_win_path)
173             expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
174
175         if self.cfg.min_free_gb > 0:
176             free = self._wsl_free_gb(self.cfg.wsl_stage_dir)

```

```

174         if free is not None and free < self.cfg.min_free_gb:
175             logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
176                           "consider running tools/cleanup_caches.py.",
177                           free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
178
179         cmd = (
180             f"conda activate {self.cfg.conda_env} && "
181             f"cd {self.cfg.repo_dir}/src && "
182             f"python predict.py "
183             f"--video_path {wsl_tilde_quote(wsl_video)} "
184             f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
185             f"--output_dir {wsl_tilde_quote(wsl_out)} "
186             f"--queue_length {self.cfg.queue_length}"
187         )
188         logger.info("Running TrackNet inference on %s ...", base)
189         try:
190             res = self._wsl_bash(cmd)
191         except subprocess.TimeoutExpired:
192             logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
193             return None
194
195         if res.returncode != 0:
196             logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
197             return None
198
199         if not os.path.exists(expected_csv_win):
200             logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
201             return None
202         logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
203
204         # Success ? the CSV is the durable output; drop the staged video copy (a pure,
205         # regenerable intermediate). On earlier failure we return above without
cleaning.
206         if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
207             logger.info("Cache hygiene: removing staged video for %s "
208                       "(set indexing.tracknet.keep_frames=true to retain).", key)
209             self._wsl_rm_rf([wsl_video])
210             return expected_csv_win
211
212     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
213         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
214
215         Returns the WSL path to the staged copy, or None on failure.
216         """
217         src_mnt = to_wsl_mnt_path(video_win_path)
218         dest = f"{self.cfg.wsl_stage_dir}/{base}"
219         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
220         try:
221             res = self._wsl_bash(cmd)
222         except subprocess.TimeoutExpired:
223             logger.error("Staging video into WSL timed out.")
224             return None
225         if res.returncode != 0 or "OK" not in (res.stdout or ""):
226             logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
227             return None
228         return dest
229
230     # ----- CSV -> windows
231
232     @staticmethod

```

```

233 def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
234     """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
235     points: List[TrajectoryPoint] = []
236     with open(csv_win_path, newline="") as f:
237         reader = csv.DictReader(f)
238         for row in reader:
239             try:
240                 points.append(TrajectoryPoint(
241                     frame=int(row["Frame"]),
242                     visible=int(row["Visibility"]) == 1,
243                     x=float(row["X"]),
244                     y=float(row["Y"]),
245                 ))
246             except (KeyError, ValueError):
247                 continue
248     points.sort(key=lambda p: p.frame)
249     return points
250
251 @staticmethod
252 def trajectory_to_action_windows(
253     points: List[TrajectoryPoint],
254     fps: float,
255     frame_width: float,
256     chunk_start: float = 0.0,
257     velocity_thresh: float = 0.02,
258     merge_gap: float = 2.0,
259     min_window_duration: float = 2.0,
260     chunk_index: Optional[int] = None,
261 ) -> List[Dict[str, Any]]:
262     """Convert a shuttle trajectory into candidate rally windows.
263
264     A frame is "active" when the shuttle is visible AND its inter-frame
265     displacement (normalised by frame width, per second) exceeds
266     ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
267     ``merge_gap`` seconds of each other are grouped into one window; short
268     windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
269     shape feeding the AI-handoff phase is identical.
270     """
271     if fps <= 0:
272         fps = 30.0
273     if frame_width <= 0:
274         frame_width = 1280.0
275
276     # Compute per-frame normalised velocity from the last *visible* point.
277     active = [] # (frame_idx, velocity)
278     prev: Optional[TrajectoryPoint] = None
279     for p in points:
280         if not p.visible:
281             prev = None # break the trajectory; absent shuttle = not in flight
282             continue
283         if prev is not None:
284             dframes = p.frame - prev.frame
285             if dframes > 0:
286                 dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
287                 velocity = (dist / frame_width) * (fps / dframes)
288                 if velocity > velocity_thresh:
289                     active.append((p.frame, velocity))
290             prev = p
291
292     if not active:
293         return []
294
295     max_gap_frames = int(merge_gap * fps)
296
297     def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:

```

```

298         vels = [v for _, v in group]
299         return {
300             "start": group[0][0] / fps + chunk_start,
301             "end": group[-1][0] / fps + chunk_start,
302             "active_frame_count": len(group),
303             "peak_velocity": max(vels),
304             "mean_velocity": sum(vels) / len(vels),
305             "track_id_count": 1, # single shuttle; kept for window-shape parity
306             "chunk_index": chunk_index,
307         }
308
309     windows = []
310     group = [active[0]]
311     for af in active[1:]:
312         if af[0] - group[-1][0] <= max_gap_frames:
313             group.append(af)
314         else:
315             windows.append(_finalize(group))
316             group = [af]
317     windows.append(_finalize(group))
318
319     return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
320

```

11. assistant (2026-06-13T19:14:14.800Z)

Now I have the critical evidence. Let me analyze the WASB inference pipeline's coordinate space carefully.

In ``wasb_infer.py``, the inference pipeline uses ``ImageDataset(cfg, remaining, input_wh, output_wh, ...)`` where ``input_wh = (inp_width, inp_height)`` and ``output_wh = (out_width, out_height)`` from the model config. This is the key ? WASB's ``ImageDataset`` resizes frames to the model's ``input_wh``, runs the heatmap detector which outputs at ``output_wh``, and the detector's ``run_tensor`` returns coordinates. The question is: in what coordinate space are the X,Y values written to the CSV?

The detector outputs ``det["xy"]`` at the heatmap/``output_wh`` scale, and the tracker passes those through. WASB-SBDT's detector/tracker typically scale detections back to the original frame resolution using the affine transform ``trans`` (note ``detector.run_tensor(imgs, trans)`` passes the transform). Let me verify the documented coordinate-space behavior and check what the fusion features docstring/plan say about this, plus look at the config and the memory note about the proxy-index recipe.

12. user (2026-06-13T19:14:17.362Z)

```

3: description: "Running the WASB GPU pipeline E2E on a big 4K video: Modern Standby kills long
runs; cv2 frame extraction is the bottleneck; the proxy+keepawake+ffmpeg-preextract recipe."
34: 3. **4K is decode-bound for WASB.** Downscale to a 1080p PROXY first (GPU `h264_nvenc`, ~4.7x
35:  realtime), index the proxy, but STITCH the final reel from the 4K ORIGINAL ? timestamps
transfer
36: 1:1 (identical timeline). The runner I used: `scratch/run_gx010125.py` (INDEX_PATH=proxy,
37:  SOURCE_PATH=4K). 1080p is also the resolution all golden evals used.

```

13. user (2026-06-13T19:14:18.600Z)

```

29: | `motion` | raw pixel motion | `backend/pipeline/segmenters/motion.py` |
70: court cam.* But we keep **the shuttle as the camera-invariant primary** so the general bet
stays robust
91: - **Shuttle = primary, camera-invariant.** Windows still **seed** from
123: The cues emit a **small set of aggregated, scale/translation-invariant per-window features ?
NOT raw
124: coordinates.** Raw pixel coordinates would memorize *this* court/camera and fail to
generalize off a
139: | `shuttle_player_colocation` | fraction of frames the shuttle sits inside/adjacent to a

```

```

person box (hand region) | **held = high; in-flight = mostly free space** ? the direct held-
shuttle discriminator |
147:window with 0?1 in-court players / one-sided motion / high shuttle-colocation (warm-up,
floor shuttle,
176:| `shuttle_player_colocation` | low (free space) | high (near a hand) | **NEW** (needs the
person cue) |
180:So the in-flight-vs-held fix is **mostly Gate A (already built) + the new colocation feature
from the
209:**Box-first.** Occupancy + count + two-sided motion + shuttle-colocation is enough for both
the gate
274:- **P3 ? feature-level (learned) fusion:** add the person features (incl.
`shuttle_player_colocation`)
288: person-colocation) with **synthetic trajectories + person boxes**. A key-license assertion
test

```

14. assistant (2026-06-13T19:14:25.411Z)

This confirms the proxy-index recipe exists exactly as the finding describes: in `gotcha-long-runs-gpu-wsl.md` lines 34-37, the recommended recipe is to "Downscale to a 1080p PROXY first... index the proxy, but STITCH the final reel from the 4K ORIGINAL". So WASB runs on the proxy while... let me verify whether the person detector in the fusion path would read the proxy or the original. This is the load-bearing question.

In `fusion\_hybrid.py:98`, `det = self.\_person.detections\_per\_frame(video\_path, ...)` ? the `video\_path` passed is the same one the segmenter operates on. Let me check what path the trajectory hybrid passes to `\_enrich\_candidate\_stats` and whether index/source paths can diverge in the pipeline.

15. user (2026-06-13T19:14:26.161Z)

```

1      """Shared engine for trajectory-substrate hybrid segmenters.
2
3      tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4      standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5      now the single home for the whole pipeline shape:
6
7          healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8          -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10     A concrete subclass supplies only its identity and runner wiring:
11     - ``family`` ? config section under ``indexing.*``, output subdir, and
12       the ``<family>-hybrid-<model>`` detector tag in metadata.
13     - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14     - ``name`` / ``description`` properties (the registry contract).
15     - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16       detection_params extras such as weights_path).
17
18     This base is deliberately NOT registered; only concrete subclasses register.
19     """
20     import os
21     import time
22     import logging
23     import concurrent.futures
24     from typing import Any, Dict, List, Optional, Tuple
25
26     import cv2
27
28     from backend.pipeline.segmenters.base import BasePlaySegmenter
29     from backend.utils.video import get_video_duration, extract_video_chunk
30     from backend.sports import sport_registry
31     from backend.providers import provider_registry
32     from backend.pipeline.detectors.base import DetectorRunner
33
34     logger = logging.getLogger(__name__)
35

```

```

36
37     def _fmt_ts(seconds: float) -> str:
38         seconds = max(0, int(seconds))
39         h, rem = divmod(seconds, 3600)
40         m, s = divmod(rem, 60)
41         return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44     class TrajectoryHybridSegmenter(BasePlaySegmenter):
45         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46         boundaries."""
47
48         #: config section under `indexing` (velocity_threshold lives there), the
49         #: output subdir for trajectories, and the metadata detector-tag prefix.
50         family: str = ""
51         #: human-readable label used in log lines.
52         display_label: str = ""
53
54         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55         Any]]:
56             """Return (runner, detector-specific detection_params extras)."""
57             raise NotImplementedError
58
59         @staticmethod
60         def _probe_fps_width(video_path: str) -> tuple:
61             """Return (fps, frame_width) for a video, with sensible fallbacks."""
62             cap = cv2.VideoCapture(video_path)
63             fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
64             width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
65             cap.release()
66             return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
67
68         @staticmethod
69         def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
70             """Provider-reported exchange count, else a duration-based estimate.
71
72             One canonical implementation ? the twins had drifted (wasb relied on
73             ``int(None)`` raising into the fallback; same result, by accident).
74             """
75             shot_count = segment.get("shuttle_exchange_count")
76             if shot_count is None:
77                 return max(3, int(duration / 0.8))
78             try:
79                 return int(shot_count)
80             except (ValueError, TypeError):
81                 return max(3, int(duration / 0.8))
82
83         # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
84         -----
85         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
86         frame_width: float, video_path: str,
87         idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
88             """Stats dict persisted into ``candidate_segments.stats`` for one window. The
89             BASE
90             returns exactly the 4 motion keys, so the trajectory twins persist byte-
91             identical
92             rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
93             features). ``points`` are the whole-video trajectory points
94             (TrajectoryPoint)."""
95             return {
96                 "peak_velocity": cw["peak_velocity"],
97                 "mean_velocity": cw["mean_velocity"],
98                 "active_frame_count": cw["active_frame_count"],
99                 "track_id_count": cw["track_id_count"],
100             }

```

```

95
96     def _select_for_handoff(self, windows: List[Dict[str, Any]],
97                             window_stats: List[Dict[str, Any]],
98                             idx_cfg: Dict[str, Any]) -> List[int]:
99         """Indices of the candidate windows that proceed to the AI handoff (and thus may
100         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
101         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
102         Persisted candidates are NOT affected ? they remain the full superset for
offline
103         re-scoring."""
104         return list(range(len(windows)))
105
106     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
107                      player_pool: Optional[List[str]] = None,
108                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
109         # override-ignore: the ABC declares the Result contract (Success|Failure)
110         # but every live segmenter still returns a bare list ? the documented
111         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
112         label = self.display_label
113         # --- Sport profile + AI provider wiring (identical across segmenters) ---
114         sport_name = self.config.get("sport", "badminton")
115         try:
116             sport_profile = sport_registry.get(sport_name)
117         except KeyError as e:
118             logger.error(str(e))
119         return []
120
121         idx_cfg = self.config.get("indexing", {})
122         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
123         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
124
125         api_key_env_var = f"{provider_name.upper()}_API_KEY"
126         api_key = os.environ.get(api_key_env_var)
127         if not api_key:
128             logger.error(
129                 f"{api_key_env_var} environment variable is not set! "
130                 f"To run the {provider_name} handoff, set your API key. "
131                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
132             )
133             return []
134         try:
135             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
136         except KeyError as e:
137             logger.error(str(e))
138         return []
139
140         video_duration = get_video_duration(video_path)
141         if video_duration <= 0:
142             logger.error("Invalid video duration.")
143             return []
144         fps, frame_width = self._probe_fps_width(video_path)
145
146         # --- Runner config + windowing thresholds ---
147         runner, runner_params = self._build_runner(idx_cfg)
148
149         velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
150         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
151         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
152         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
153         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
154         log_candidates = idx_cfg.get("log_yolo_candidates", True)
155         store_candidates = idx_cfg.get("store_yolo_candidates", True)

```

```

156
157     detection_params = {
158         "detector": self.name,
159         "velocity_thresh": velocity_thresh,
160         "merge_gap": merge_gap,
161         "min_action_window_duration": min_window_duration,
162         **runner_params,
163     }
164
165     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
166     ok, msg = runner.healthcheck()
167     if not ok:
168         logger.error("%s WSL environment not ready: %s", label, msg)
169         return []
170     logger.info("%s WSL env OK: %s", label, msg)
171
172     # --- Phase 2: trajectory -> candidate rally windows ---
173     # Trajectory detectors process the whole video in one pass (they carry
174     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
175     t_start = time.time()
176     out_dir = os.path.join("output", self.family)
177     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
178     if not csv_path:
179         logger.error("%s produced no trajectory; aborting.", label)
180         return []
181
182     points = runner.parse_trajectory_csv(csv_path)
183     logger.info("Parsed %d trajectory points (%d visible).",
184                 len(points), sum(1 for p in points if p.visible))
185
186     all_candidate_windows = runner.trajectory_to_action_windows(
187         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
188         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
189         min_window_duration=min_window_duration, chunk_index=0,
190     )
191     logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
192                 label, time.time() - t_start, len(all_candidate_windows))
193
194     if log_candidates:
195         for cw in all_candidate_windows:
196             logger.info(f"[{label}] rally
197 {fmt_ts(cw['start'])}-{fmt_ts(cw['end'])} "
198 f"({cw['end'] - cw['start']:.1f}s) |
199 frames={cw['active_frame_count']} "
200 f"peak_v={cw['peak_velocity']:.3f}
201 mean_v={cw['mean_velocity']:.3f}")
202
203     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
204     fusion
205     # segmenter adds net_crossings + person-cue features. Computed once, reused for
206     both
207     # persistence and the handoff gate below.
208     window_stats = [
209         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
210         idx_cfg)
211         for cw in all_candidate_windows
212     ]
213
214     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
215     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
216     if store_candidates:
217         for i, cw in enumerate(all_candidate_windows):
218             candidate_ids[i] = self.db.add_candidate_segment(
219                 video_id, detector=self.name,
220                 start_time=cw["start"], end_time=cw["end"],

```



```

215             chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"])),
216             stats=window_stats[i], detection_params=detection_params,
217         )
218
219     if not all_candidate_windows:
220         return []
221
222     # --- Phase 3: AI provider handoff over padded candidate clips ---
223     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
224     # candidates above stay the full superset ? only the served play_segments are
gated.
225     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
226     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
227     os.makedirs(handoff_dir, exist_ok=True)
228     logger.info("Phase 3: %s validation on %d candidate windows...",
229               provider_name, len(handoff_indices))
230
231     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
232         w_start = max(0, window["start"] - handoff_padding)
233         w_end = min(video_duration, window["end"] + handoff_padding)
234         w_dur = w_end - w_start
235         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
236         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
237             return []
238         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
239         segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
240                                           label=f"Handoff {wi+1}")
241         valid = []
242         for seg in segments:
243             try:
244                 seg["start_time"] = float(seg["start_time"]) + w_start
245                 seg["end_time"] = float(seg["end_time"]) + w_start
246                 seg["_candidate_id"] = candidate_ids[wi]
247                 valid.append(seg)
248             except (ValueError, TypeError, KeyError) as e:
249                 logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
250             try:
251                 os.remove(w_path)
252             except OSError:
253                 pass
254         return valid
255
256     ai_segments: List[dict] = []
257     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
258         futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
259                   for wi in handoff_indices]
260         for future in futures:
261             ai_segments.extend(future.result())
262         try:
263             os.rmdir(handoff_dir)
264         except OSError:
265             pass
266         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
267
268     # --- Phase 4: DB population + candidate linking ---
269     segments_data = []
270     for segment in ai_segments:

```

```

271         try:
272             start = float(segment["start_time"])
273             end = float(segment["end_time"])
274             dur = end - start
275             metadata = {
276                 "shot_count": self._shot_count_from(segment, dur),
277                 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
278                 "detector": f"{self.family}-hybrid-{model_name}",
279             }
280             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
281             if segment_id:
282                 candidate_id = segment.get("_candidate_id")
283                 if candidate_id is not None:
284                     self.db.link_candidate_to_segment(candidate_id, segment_id)
285                 segments_data.append({
286                     "id": segment_id, "start_time": start, "end_time": end,
287                     "duration": dur, "metadata": metadata,
288                 })
289             except Exception as e:
290                 logger.warning("Skipping invalid segment struct: %s", e)
291
292         return segments_data
293

```

16. assistant (2026-06-13T19:14:40.517Z)

This is the decisive piece of evidence. In `trajectory\_hybrid.py`, the SAME `video\_path` is threaded through the entire pipeline:

- Line 177: `csv\_path = runner.run\_predict(video\_path, ...)` ? WASB runs on `video\_path`
- Line 204: `self.\_enrich\_candidate\_stats(cw, points, fps, frame\_width, video\_path, idx\_cfg)` ? person detector reads the SAME `video\_path`

So within a single `process\_video` call, the shuttle trajectory and the person boxes are ALWAYS derived from the **same** `video\_path` at the same resolution. There is no place in this code where the segmenter is given a separate "index path" vs "source path."

The proxy-index recipe described in memory (`scratch/run\_gx010125.py` with `INDEX\_PATH=proxy`, `SOURCE\_PATH=4K`) is a **scratch-runner-level** orchestration: the proxy is used for indexing (segmentation) and the 4K original is used only for STITCHING the final reel. The segmenter itself only ever sees one path (the index path = proxy). The person detector would also read that same proxy path. Let me confirm the scratch runner actually splits these two paths and that stitching is a separate stage that does not feed the person detector.

17. user (2026-06-13T19:14:40.973Z)

```

1         """One-off non-interactive E2E runner for GX010125.MP4 (modeled on
run_process_and_stitch.py).
2
3         Every knob ? segmenter, min_shot_count, paddings, watermark ? comes from config.json.
4         Nothing is overridden here, so the run demonstrates the config is actually picked up.
5         """
6         import json
7         import logging
8         import os
9         import sys
10        import time
11
12        sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__)))) # repo
root
13
14        from dotenv import load_dotenv
15
16        load_dotenv()

```

```

17
18     logging.basicConfig(
19         level=logging.INFO,
20         format="%(asctime)s [%(name)s] %(levelname)s: %(message)s",
21         datefmt="%H:%M:%S",
22     )
23
24     from backend.infrastructure.database import Database
25     from backend.config import load_config
26     from backend.utils.hashing import compute_video_id
27     from backend.pipeline.segmenters import segmenter_registry
28     from backend.pipeline.stitcher.compiler import VideoCompiler
29
30     # Segmentation runs on the 1080p proxy (4K60 decode was the bottleneck ? measured ~5 fps
31     # frame extraction = ~2.5h for the trajectory pass). Timestamps transfer 1:1 (same
32     # timeline), so the final reel is cut from the ORIGINAL 4K for full quality.
33     INDEX_PATH = r"C:\Users\avidu\Projects\badminton-highlight-
indexer\output\GX010125_1080p.mp4"
34     SOURCE_PATH = r"F:\GoPro Hero Black 12\KhelSutra\Badminton 12 June 2026\GX010125.MP4"
35     OUTPUT_FILENAME = "GX010125_highlights.mp4"
36
37
38     def main():
39         cfg = load_config()
40         print(f"[CONFIG] segmenter={cfg.indexing.default_segmenter} | "
41             f"min_shot_count={cfg.stitching.min_shot_count} | "
42             f"begin_padding={cfg.stitching.begin_padding}s
end_padding={cfg.stitching.end_padding}s | "
43             f"watermark={'on' if cfg.stitching.watermark.enabled else 'off'}", flush=True)
44
45         for p in (INDEX_PATH, SOURCE_PATH):
46             if not os.path.exists(p):
47                 raise SystemExit(f"video not found: {p}")
48
49         os.makedirs(cfg.output.output_dir, exist_ok=True)
50         db = Database(os.path.join(cfg.output.output_dir, cfg.output.db_name))
51
52         t0 = time.time()
53         print("[HASH] computing MD5 of the 1080p proxy...", flush=True)
54         video_id = compute_video_id(INDEX_PATH)
55         print(f"[HASH] video_id={video_id} ({time.time() - t0:.0f}s)", flush=True)
56
57         segments = []
58         cached = db.get_video(video_id)
59         if cached and cached.get("status") == "processed":
60             segments = db.query_play_segments(video_id, {})
61             if segments:
62                 print(f"[CACHE] reusing {len(segments)} cached segments for {video_id}",
flush=True)
63
64         if not segments:
65             print("[INDEX] clean indexing (validators bypassed for the massive file, "
66                 "as in run_process_and_stitch.py)...", flush=True)
67             db.clear_video_data(video_id)
68             db.add_video(video_id, INDEX_PATH, "processed", [])
69             seg_name = cfg.indexing.default_segmenter
70             segmenter_cls = segmenter_registry.get(seg_name)
71             if not segmenter_cls:
72                 raise SystemExit(f"segmenter '{seg_name}' not registered")
73             segmenter = segmenter_cls(db, cfg)
74             segments = segmenter.process_video(video_id, INDEX_PATH)
75             print(f"[INDEX] {len(segments)} play segments indexed "
76                 f"({time.time() - t0:.0f}s).", flush=True)
77
78         if not segments:

```

```

79         raise SystemExit("no play segments detected ? nothing to stitch")
80
81     min_shots = cfg.stitching.min_shot_count
82     time_pairs, skipped = [], 0
83     for s in segments:
84         meta = s.get("metadata") or {}
85         if isinstance(meta, str):
86             try:
87                 meta = json.loads(meta)
88             except Exception:
89                 meta = {}
90         sc = meta.get("shot_count", 0)
91         line = (f" segment id={s.get('id')} "
92               f"[{float(s['start_time']):8.2f}s - {float(s['end_time']):8.2f}s] "
93               f"shot_count={sc} confidence={meta.get('shot_count_confidence', '?')}")
94         if sc >= min_shots:
95             time_pairs.append((s["start_time"], s["end_time"]))
96             print(line + " -> KEEP", flush=True)
97         else:
98             skipped += 1
99             print(line + " -> drop", flush=True)
100
101     print(f"[FILTER] config min_shot_count={min_shots}: "
102          f"kept {len(time_pairs)}, dropped {skipped}", flush=True)
103     if not time_pairs:
104         raise SystemExit(f"no segments passed the min_shot_count={min_shots} filter")
105
106     compiler = VideoCompiler(cfg.output.output_dir,
107                              begin_padding=cfg.stitching.begin_padding,
108                              end_padding=cfg.stitching.end_padding,
109                              watermark=cfg.stitching.watermark)
110     out = compiler.compile_highlights(SOURCE_PATH, time_pairs, OUTPUT_FILENAME) # cut
111     print(f"[DONE] highlight reel: {os.path.abspath(out)} (total {time.time() -
112     t0:.0f}s)", flush=True)
113
114     if __name__ == "__main__":
115         main()
116

```

18. assistant (2026-06-13T19:14:54.945Z)

This is conclusive. The proxy-index recipe in `scratch/run\_gx010125.py`:

- Line 74: `segments = segmenter.process\_video(video\_id, INDEX\_PATH)` ? the segmenter (and therefore BOTH the WASB runner AND the person detector) receive `INDEX\_PATH` (the 1080p proxy).
- Line 110: `compiler.compile\_highlights(SOURCE\_PATH, ...)` ? only the STITCHER uses the 4K `SOURCE\_PATH`.

So even in the proxy-index mode, the person detector reads the SAME proxy that WASB ran on (both go through `process\_video(video\_id, INDEX\_PATH)`). The stitcher's use of the 4K original is downstream and entirely decoupled from feature computation. The two pixel spaces do NOT diverge in this recipe.

Now let me assess the finding's core claim fairly. The finding acknowledges this is "currently latent (not triggered by the default path)." Let me re-read its precise claim to see if it holds even as a latent risk:

The finding says: "the memory notes a 'proxy-index' recipe where WASB runs on a downscaled proxy while the person detector reads the full-res original. In that mode the two pixel spaces diverge."

This specific claim is ****factually wrong**** about how the proxy-index recipe works. As verified above, in the proxy-index recipe both WASB and the person detector read the SAME proxy (`INDEX\_PATH`); the original is used only for stitching. There is no code path ? neither default

nor the proxy recipe ? where WASB and the person detector are fed different-resolution inputs. The person detector and WASB are both driven from the single `video\_path` argument to `process\_video`, which is structurally guaranteed to be one path.

Let me also verify the secondary technical claim: whether the WASB CSV coordinates are even in the cv2-frame pixel space (i.e., original-frame coords) or some model-internal space. The detector receives the affine `trans` (line 297 in wasb_infer.py: `detector.run\_tensor(imgs, trans)`), which is WASB's mechanism to map detections back to original frame coordinates. And `extract\_frames` (wasb_infer.py:348-376) writes frames at native `cap.read()` resolution (no resize), so the input frames to WASB are the proxy's native resolution and the detector maps back to that same resolution. This means the CSV X,Y are in the proxy's pixel space ? exactly the same space the person detector's cv2 reads (also the proxy). They match.

Let me do a final check on the config model to confirm there's no separate index/source path config that could split them at the API level.

19. user (2026-06-13T19:14:56.253Z)

```
diff --git a/backend/config/models.py b/backend/config/models.py
```

```
index bb56881..7f8191c 100644
```

```
--- a/backend/config/models.py
```

```
+++ b/backend/config/models.py
```

```
@@ -75,6 +75,39 @@ class WasbIndexConfig(BaseModel):
```

```
    min_free_gb: float = 0.0
```

```
+class FusionConfig(BaseModel):
```

```
+    """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
```

```
+    """
```

```
+    Every default reproduces control behaviour: the fusion segmenter persists the  
+    shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']`` is  
+    true ? the person-cue features, but it does NOT gate the served handoff unless a  
+    threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court mask  
+    is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-only);  
+    supplied ? off-court persons (spectators / adjacent courts) are excluded.
```

```
+    """
```

```
+    # Which trajectory substrate seeds the windows (shuttle stays primary).
```

```
+    substrate: Literal["wasb", "tracknet"] = "wasb"
```

```
+    # Windowing velocity threshold for the fusion family (the engine reads
```

```
+    # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
```

```
+    # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
```

```
+    velocity_threshold: float = 0.02
```

```
+    # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
```

```
+    # enabled ? so the default path never imports torch / touches the GPU.
```

```
+    cue_flags: dict[str, bool] = Field(default_factory=dict)
```

```
+    # Served Gate A: drop windows whose shuttle net-crossings < this from the AI handoff.
```

```
+    # 0 = OFF (no gating; persisted candidates are always the full superset for offline re-  
+    scoring).
```

```
+    min_crossings: int = Field(default=0, ge=0)
```

```
+    # Served Gate B: when the person cue is on, drop windows with median in-court players
```

```
+    # < this. 0 = OFF.
```

```
+    min_players: int = Field(default=0, ge=0)
```

```
+    # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
```

```
+    court_polygon: Optional[list[list[float]]] = None
```

```
+    # Net line for the person two-sided-motion split (person features only ? the shuttle
```

```
+    # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
```

```
+    net_axis: Literal["x", "y"] = "y"
```

```
+    net_position: float = Field(default=0.5, ge=0.0, le=1.0)
```

```
+    """
```

```
+    """
```

```
+class IndexingConfig(BaseModel):
```

```
    default_segmenter: str = "yolo_hybrid"
```

```
    use_gpu: bool = True
```

```
@@ -106,6 +139,7 @@ class IndexingConfig(BaseModel):
```

```

    tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
    wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
+   fusion: FusionConfig = Field(default_factory=FusionConfig)

    @field_validator("default_segmenter")
    @classmethod

```

20. assistant (2026-06-13T19:15:16.037Z)

The `FusionConfig` has no separate index-path vs source-path setting ? confirming there is no configuration knob that could feed WASB and the person detector different-resolution inputs. The substrate (`wasb`/`tracknet`) and the person detector both operate on the single `video\_path` passed to `process\_video`.

I now have a complete picture. Let me render my verdict.

Verdict

****The finding is partially true on the abstract observation but materially wrong on its central concrete claim, and its severity is overstated even as a "low".****

****What's true:**** `shuttle\_player\_colocation` / `\_shuttle\_in\_box` (fusion_features.py:154-185) do compare shuttle pixel coords directly against person-box pixel coords, and this is only correct if both live in the same pixel space. The shuttle coords come from the WASB CSV (`parse\_trajectory\_csv`: `x=float(row["X"])\`, tracknet_runner.py:243) and the person boxes from cv2 (`detections\_per\_frame`, person_detector.py:282-283). There is no scale conversion or assertion. That part is accurate. The colocation feature is also genuinely NOT normalized while the other features are (foot/center/motion divide by `fw`/`fh`) ? that asymmetry is real.

****What refutes the finding's core claim:**** The finding's load-bearing argument ? that the proxy-index recipe is "a mode where WASB runs on a downscaled proxy while the person detector reads the full-res original" so "the two pixel spaces diverge" ? is ****factually false****. I traced the actual proxy-index recipe:

- `trajectory\_hybrid.py:177` runs WASB on `video\_path`, and `:204` runs the person detector (via `\_enrich\_candidate\_stats` ? `\_person\_features` ? `detections\_per\_frame`) on the ****same**** `video\_path`. Both signals are structurally bound to one path within a `process\_video` call.
- The proxy recipe (`scratch/run\_gx010125.py:74`) calls `segmenter.process\_video(video\_id, INDEX\_PATH)` with the 1080p proxy ? so ****both**** WASB and the person detector read the proxy. The 4K `SOURCE\_PATH` is used ****only**** by the stitcher (`:110`, `compile\_highlights`), which is fully decoupled from feature computation.
- `FusionConfig` (config diff) exposes no index-vs-source path split.

So the divergence scenario the finding describes does not exist in any current code path ? not the default, and not the proxy recipe it cites. The person detector and WASB are fed the same bytes by construction. WASB also maps detections back to its input-frame resolution (the affine `trans` at wasb_infer.py:297; `extract\_frames` decodes at native resolution with no resize), so the CSV coords are in the proxy's pixel space ? identical to what cv2 reads.

****Net:**** This is a latent code-smell / documentation-hygiene observation (the colocation feature would break if a *future, hypothetical* refactor split the two paths), not a real coupling bug, and the specific failure mode it names is mischaracterized. It is a defensible "nice-to-have invariant comment," but as written it asserts a concrete divergence that the code does not have. Marking `real=false` per the adversarial-verification bar: the headline coupling is non-existent in the actual code, and the supporting "proxy-index" rationale is wrong.

21. user (2026-06-13T19:15:27.375Z)

Structured output provided successfully

22. assistant (2026-06-13T19:15:34.334Z)

I've completed the adversarial verification.

The finding is **not real (real=false)**. While its abstract observation about pixel-space comparison is accurate, its load-bearing concrete claim ? that the proxy-index recipe creates a mode where WASB runs on a downscaled proxy while the person detector reads the full-res original ? is factually wrong. I traced the actual code paths:

- ``backend/pipeline/segmenters/trajectory_hybrid.py:177`` and ``:204`` thread the **same** ``video_path`` to both WASB (``run_predict``) and the person detector (``_enrich_candidate_stats`` ? ``_person_features`` ? ``detections_per_frame``).
- The actual proxy recipe (``scratch/run_gx010125.py:74``) calls ``process_video(video_id, INDEX_PATH)`` with the 1080p proxy, so **both** signals read the proxy; the 4K ``SOURCE_PATH`` is used only by the stitcher (``:110``), fully decoupled from feature math.
- ``FusionConfig`` exposes no index-vs-source path split.

The two pixel spaces are bound to one path by construction and cannot diverge in either the default or the proxy mode. At most this is a latent documentation nicety about a hypothetical future refactor, not a real coupling bug ? and the specific failure mode it cites is mischaracterized.